



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN
LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 0

NOMBRE COMPLETO: HERNÁNDEZ FIERROS SONIA

N° de Cuenta: 319194359

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 07

SEMESTRE: 2027-1

FECHA DE ENTREGA LÍMITE: 9 de septiembre del 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

```
359 //Triangulo 4
360 0.7f, 0.5f, 0.0f, 0.6f, 0.2f, 0.8f,
361 0.7f, 0.4f, 0.0f, 0.6f, 0.2f, 0.8f,
362 0.5f, 0.4f, 0.0f, 0.6f, 0.2f, 0.8f,
363 //Triangulo 5
364 0.5f, 0.3f, 0.0f, 0.6f, 0.2f, 0.8f,
365 0.7f, 0.3f, 0.0f, 0.6f, 0.2f, 0.8f,
366 0.5f, 0.2f, 0.0f, 0.6f, 0.2f, 0.8f,
367 //Triangulo 6
368 0.5f, 0.2f, 0.0f, 0.6f, 0.2f, 0.8f,
369 0.7f, 0.2f, 0.0f, 0.6f, 0.2f, 0.8f,
370 0.7f, 0.3f, 0.0f, 0.6f, 0.2f, 0.8f
371 };
372 MeshColor* letraF = new MeshColor();
373 letraF->CreateMeshColor(vertices_F, sizeof(vertices_F) / sizeof(GLfloat));
374 meshColorList.push_back(letraF);
375
376 AgregarCuboColor(glm::vec3(0.0f, 0.0f, 1.0f)); // [3] cubo azul
377 AgregarCuboColor(glm::vec3(0.478f, 0.255f, 0.067f)); // [4] cubo cafe
378 AgregarCuboColor(glm::vec3(0.0f, 0.0f, 0.0f)); // [5] cubo negro
379 AgregarPiramideTriangularColor(glm::vec3(1.0f, 1.0f, 0.0f)); // [6] piramide amarilla
380 AgregarPiramideTriangularColor(glm::vec3(0.0f, 1.0f, 0.0f)); // [7] piramide verde
381 AgregarPiramideTriangularColor(glm::vec3(1.0f, 0.0f, 0.0f)); // [8] piramide roja
382 AgregarPiramideTriangularColor(glm::vec3(0.6f, 0.2f, 0.8f)); // [9] piramide morada
383 AgregarTrianguloRectoColor(glm::vec3(1.0f, 1.0f, 0.0f)); // [10] esquina amarilla
384 AgregarTrianguloRectoColor(glm::vec3(1.0f, 0.0f, 0.0f)); // [11] esquina roja
385 AgregarTrianguloRectoColor(glm::vec3(0.0f, 1.0f, 0.0f)); // [12] esquina verde
386 AgregarTrianguloRectoColor(glm::vec3(0.6f, 0.2f, 0.8f)); // [13] esquina morada
387 }
```

En esta parte se agregaron las figuras que se utilizaron para formar los ejercicios. Se reutilizaron cubos, pirámides y triángulos, asignándoles diferentes colores mediante valores RGB.

```

389 void CreateShaders()
390
391 Shader* shader1 = new Shader();
392 shader1->CreateFromFiles(vShader, fShader);
393 shaderList.push_back(*shader1); //shaderList[0]
394
395 Shader* shader2 = new Shader();
396 shader2->CreateFromFiles(vShaderColor, fShaderColor);
397 shaderList.push_back(*shader2); //shaderList[1]
398
399 Shader* shaderRojo = new Shader();
400 shaderRojo->CreateFromFiles(vShaderRojo, fShaderRojo);
401 shaderList.push_back(*shaderRojo); //shaderList[2]
402
403 Shader* shaderAmarillo = new Shader();
404 shaderAmarillo->CreateFromFiles(vShaderAmarillo, fShaderAmarillo);
405 shaderList.push_back(*shaderAmarillo); //shaderList[3]
406
407 Shader* shaderVerde = new Shader();
408 shaderVerde->CreateFromFiles(vShaderVerde, fShaderVerde);
409 shaderList.push_back(*shaderVerde); //shaderList[4]
410
411 Shader* shaderAzul = new Shader();
412 shaderAzul->CreateFromFiles(vShaderAzul, fShaderAzul);
413 shaderList.push_back(*shaderAzul); //shaderList[5]
414
415 Shader* shaderCafe = new Shader();
416 shaderCafe->CreateFromFiles(vShaderCafe, fShaderCafe);
417 shaderList.push_back(*shaderCafe); //shaderList[6]
418
419 Shader* shaderNegro = new Shader();
420 shaderNegro->CreateFromFiles(vShaderNegro, fShaderNegro);
421 shaderList.push_back(*shaderNegro); //shaderList[7]

```

Se crearon diferentes shaders para poder utilizar distintos colores en las figuras. Cada shader se relaciona con un archivo vertex y un archivo fragment, y después se agrega a la lista de shaders para poder utilizarlo al momento de dibujar.

```

576 //Rombo azul
577 shaderList[1].useShader();
578 uniformModel = shaderList[1].getModelLocation();
579 uniformProjection = shaderList[1].getProjectLocation();
580 model = glm::mat4(1.0);
581 model = glm::translate(model, glm::vec3(0.0f, -0.0204f, -2.0f));
582 model = glm::rotate(model, 45 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
583 model = glm::scale(model, glm::vec3(0.36f, 0.36f, 0.06f));
584 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
585 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
586 meshColorList[3]->RenderMeshColor();

```

En esta parte se utilizaron las transformaciones geométricas para colocar cada figura en una posición determinada. También se modificó su tamaño y, en algunos casos, su rotación para conseguir la forma final de los ejercicios.

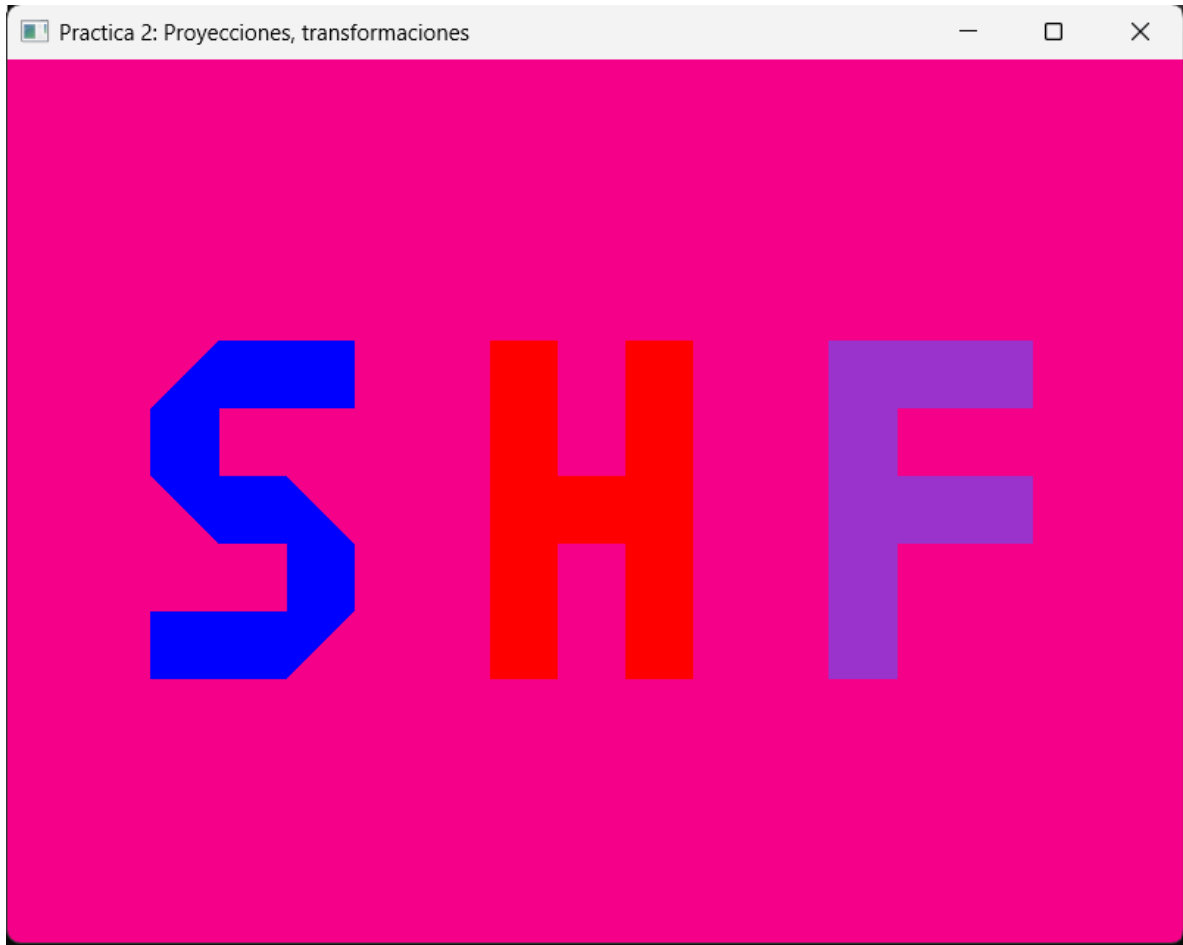
```

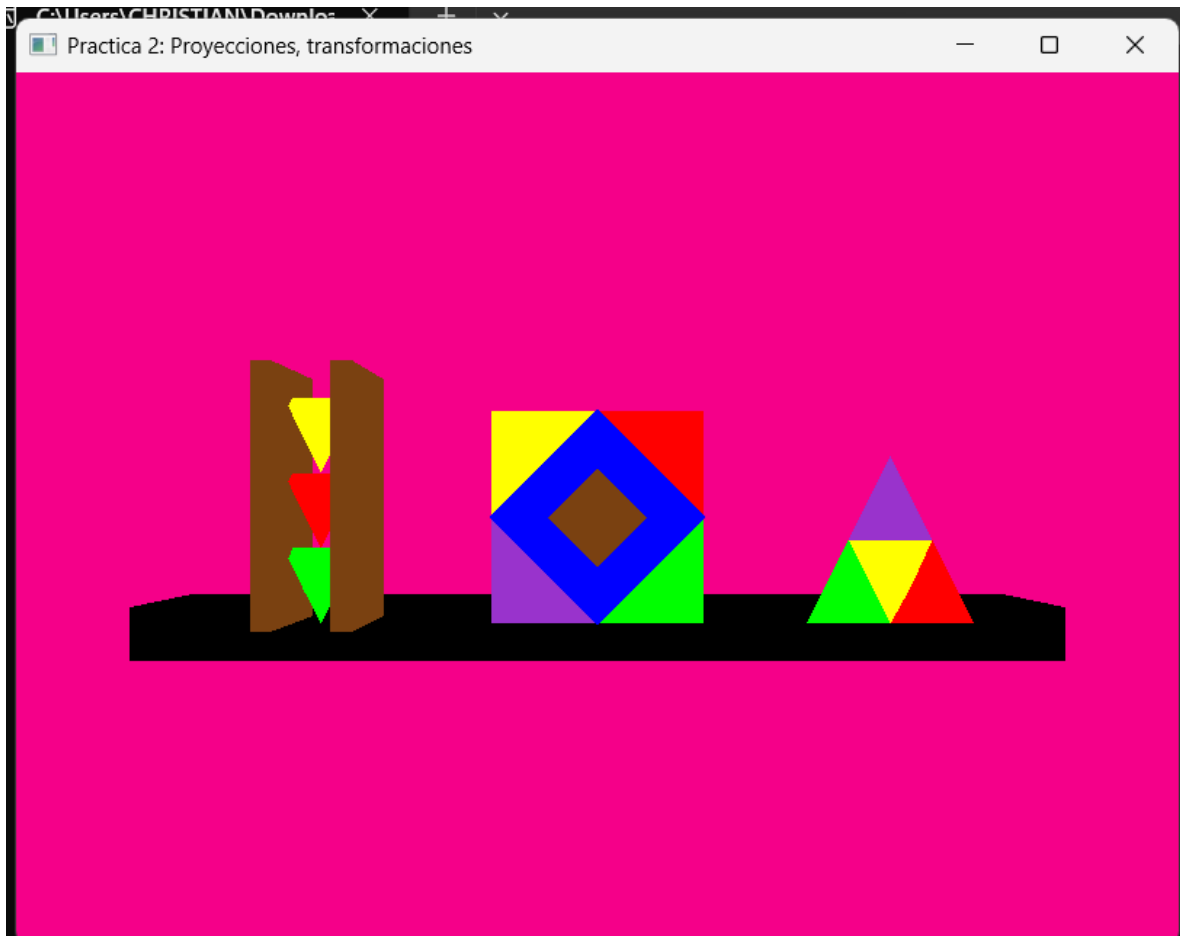
433 int main()
434 {
435     mainWindow = Window(800, 600);
436     mainWindow.Initialise();
437
438     glEnable(GL_DEPTH_TEST);
439
440     CreaPiramide();//índice 0 en MeshList
441     CrearCubo();//índice 1 en MeshList
442     CrearPiramideCuadrangular();//índice 2 en MeshList
443     CrearLetrasYFiguras();//usa MeshColor, índices en MeshColorList
444     CreateShaders();
445
446     GLuint uniformProjection = 0;
447     GLuint uniformModel = 0;
448     //Projection: Matriz de Dimensión 4x4 para indicar si vemos en 2D( orthogonal) o en 3D) perspectiva
449     //glm::mat4 projection = glm::ortho(-1.0f, 1.0f, -1.0f, 1.0f, 0.1f, 100.0f);
450     glm::mat4 projection = glm::perspective(glm::radians(55.0f), (float)mainWindow.getBufferWidth() / (float)mainWind
451
452     //Model: Matriz de Dimensión 4x4 en la cual se almacena la multiplicación de las transformaciones geométricas.
453     glm::mat4 model(1.0); //fuera del while se usa para inicializar la matriz con una identidad
454
455     //s todo lo que se crea una sola vez (ventanas, mallas, shaders, matriz de porportion si la cámara no se mueve)
456     //s lo que va adentro del main es lo que cambia da frame,
457     //Loop mientras no se cierra la ventana
458     while (!mainWindow.getShouldClose())
459     {
460         //Recibir eventos del usuario
461         glfwPollEvents();
462         //Limpiar la ventana
463         //s este es negro glClearColor(0.0f,0.0f,0.0f,1.0f);
464         glClearColor(0.9608f, 0.0f, 0.539f, 1.0f); //rosa mexicano
465         glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT); //Se agrega limpiar el buffer de profundidad
466

```

Para trabajar con las figuras en 3D se utilizó una proyección en perspectiva y se habilitó el buffer de profundidad. Esto permite que las figuras se puedan visualizar con una apariencia tridimensional y que las partes que están detrás sean tomadas en cuenta correctamente.

Finalmente, el programa muestra las diferentes figuras colocadas y transformadas de acuerdo con las indicaciones de la práctica. Se utilizaron diferentes escalas, posiciones, rotaciones y colores para obtener el resultado mostrado.





2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

Uno de los principales problemas que tuve fue entender el código general del programa, ya que al principio todavía no tenía muy claro qué partes se encargaban de crear las figuras, cuáles de los shaders y cuáles de las transformaciones.

Tampoco logré terminar a tiempo el ejercicio que se dejó durante la clase. Al principio pensé que esta práctica sería relativamente sencilla porque parecía que solamente había que pasar las figuras de 2D a 3D, pero al realizarlo me di cuenta de que no era tan sencillo, ya que las figuras podían deformarse bastante dependiendo de las transformaciones y las posiciones utilizadas.

Para resolverlo tuve que revisar nuevamente el código original y probar diferentes valores de traslación, rotación y escala hasta conseguir que las figuras tuvieran una posición y tamaño similares a los indicados.

Durante el desarrollo también se presentaron algunos resultados incorrectos en la posición y forma de las figuras. No se conservó una captura de estos errores, pero

se fueron corrigiendo modificando los valores de las transformaciones hasta obtener el resultado final.

3.- Conclusión:

a. Los ejercicios del reporte: Complejidad, Explicación.

Todo partió del código original proporcionado por el profesor, en el cual ya se tenía un cubo y un cuadrado bicolor utilizando dos shaders. A partir de ese código se reutilizaron las figuras que ya estaban definidas, como las pirámides y el cubo, y se agregaron diferentes colores y transformaciones para formar los ejercicios.

La mayor parte del trabajo consistió en colocar las figuras en diferentes posiciones y modificar su escala y rotación para obtener el resultado solicitado. También se crearon nuevos shaders para poder utilizar diferentes colores.

Las clases Mesh, Shader y Window no tuvieron que modificarse para realizar los ejercicios. De igual forma, se mantuvo el ciclo while dentro del main, ya que es el que permite que la ventana continúe funcionando y actualizándose mientras el programa está abierto.

La parte que me pareció más complicada fue entender cómo las diferentes transformaciones afectaban a las figuras, ya que un cambio pequeño en los valores podía hacer que una figura cambiara mucho de posición o incluso pareciera deformarse.

b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica

Considero que hubiera sido útil explicar algunos de los temas con un poco más de detalle y avanzar un poco más lento en algunas partes, especialmente al momento de explicar cómo se utilizan las transformaciones y cómo se relacionan las diferentes partes del código.

c. Conclusión.

Con esta práctica pude comprender un poco mejor cómo se pueden utilizar diferentes figuras, colores y transformaciones para construir una escena en 3D. Aunque al principio me costó bastante entender el código general y la forma en que se relacionaban las transformaciones, al ir modificando los valores pude entender mejor cómo afectaban a cada figura.

También pude notar que pasar de trabajar con figuras sencillas en 2D a trabajar con objetos en 3D implica tomar en cuenta más elementos, especialmente la profundidad y la proyección. En general, la práctica me ayudó a entender mejor el funcionamiento del código que se utiliza para crear y colocar objetos dentro de una escena.

1. Bibliografía en formato APA

Facultad de Ingeniería, Universidad de la República. (2011). Transformaciones geométricas en OpenGL [Presentación de diapositivas]. Instituto de Computación.
<https://www.fing.edu.uy/inco/cursos/compgraf/Clases/2011/05-Transformaciones%20Geometricas%5BopenGL%5D.pdf>